SLOT 4
DART ESSENTIALS
ASYNC PROGRAMMING IN DART & FLUTTER

I. TÓM TẮT CÁC CHỦ ĐỀ & MỤC TIÊU TRONG BÀI HỌC

- Intro to Futures & Async
- Future-based Async Programming
- Streams & StreamController

- Giúp sinh viên **hiểu đúng bản chất bất đồng bộ**, không học vẹt cú pháp
- Xóa bỏ tư duy sai: *"code không báo lỗi là đúng"*
- Tạo nền tảng **bắt buộc** cho:
  - Gọi API
  - Loading UI
  - Xử lý dữ liệu trả về
  - Realtime update
  - State Management về sau

1 FUTURES – BẤT ĐỒNG BỘ MỘT LẦN

◈ Khái niệm cốt lõi bắt buộc phải hiểu

- **Future là giá trị sẽ có trong tương lai**, không có ngay tại thời điểm gọi
- Khi gọi một hàm async:
  - Code **không đứng chờ**
  - Flutter **tiếp tục render UI**
- Một `Future` chỉ có **1 kết quả duy nhất**:
  - Thành công (`value`)
  - Thất bại (`error`)

👉 Sinh viên phải hiểu rõ:
"Future KHÔNG phải là dữ liệu, mà là **lời hứa sẽ trả dữ liệu sau**"

🎯 Mục tiêu học tập (làm được gì trong app Flutter thật)

Sau phần này, sinh viên có thể:

- Gọi API trả về dữ liệu **một lần**
- Hiển thị:
  - loading khi đang chờ
  - dữ liệu khi thành công
  - lỗi khi thất bại
- Tránh lỗi:
  - dùng dữ liệu Future như dữ liệu thường
  - UI hiển thị sai do Future chưa hoàn thành

📘 Ý nghĩa sư phạm – vì sao cực kỳ quan trọng

- **90% thao tác trong Flutter là async** (API, file, database)
- Không hiểu Future → **không hiểu vì sao UI hiển thị sai**
- Là bước đầu để sinh viên:
  - không "lo lắng" async
  - không viết code chạy lung tung nhưng không lỗi

2 ASYNC / AWAIT – VIẾT ASYNC DỄ HIỂU NHƯ ĐỒNG BỘ

◈ Khái niệm cốt lõi bắt buộc phải hiểu

- `async / await` **KHÔNG làm code chạy đồng bộ**
- Nó chỉ giúp:

PhucPT10@fpt.edu.vn

    - viết code async **dễ đọc hơn**
    - suy nghĩ theo từng bước logic
- `await` chỉ dùng được trong hàm `async`
- Khi `await`:
    - Flutter **tạm dừng hàm đó**
    - nhưng **không block UI**

👉 Sinh viên phải hiểu:
"await không làm app đứng – nó chỉ làm hàm đó chờ"

🎯 Mục tiêu học tập (làm được gì trong app Flutter thật)

Sau phần này, sinh viên có thể:

- Viết logic gọi API rõ ràng:
    - lấy dữ liệu
    - xử lý
    - cập nhật UI
- Bắt lỗi async bằng:
    - `try / catch`
- Đọc hiểu code async trong dự án thật

📝 Ý nghĩa sư phạm – vì sao cực kỳ quan trọng

- Giảm **áp lực tư duy** cho sinh viên mới
- Tránh lỗi phổ biến:
    - quên `await`
    - UI cập nhật trước khi dữ liệu có
- Là nền tảng cho:
    - Networking
    - Repository pattern
    - State Management (Bloc, Provider, Riverpod…)

3 STREAMS – DỮ LIỆU LIÊN TỤC & REALTIME

◈ THREAD TRONG ỨNG DỤNG FLUTTER (MOBILE OS)

◈ 1. Main Thread / UI Thread là gì?

- Là **luồng chính** của ứng dụng mobile (Android / iOS)
- Chịu trách nhiệm:
    - vẽ giao diện (UI)
    - xử lý thao tác người dùng (tap, scroll)
    - hiển thị animation
- 👉 **Chỉ có 1 UI Thread duy nhất**

👄 Nếu UI Thread bị chặn (block):

- App bị **đơ**
- Không cuộn được
- Dễ bị hệ điều hành **đóng app (ANR / freeze)**

◈ 2. Work Thread là gì?

- Là các luồng **xử lý công việc nặng**
- Dùng cho:
    - gọi API
    - đọc file
    - tính toán lâu
    - xử lý dữ liệu
- 👉 Work Thread **không vẽ UI**

PhucPT10@fpt.edu.vn

3. Flutter & Dart làm việc với Thread như thế nào?

- Flutter:
  - UI luôn chạy trên **UI Thread**
- Dart:
  - dùng **async / await** để:
    - đưa việc nặng ra ngoài UI Thread
    - **không block UI**
- ❗ async **KHÔNG tạo thread mới**
  - async chỉ nói với Dart:
    "việc này làm lâu → đợi kết quả sau"

Khái niệm cốt lõi bắt buộc phải hiểu

- **Stream là dòng dữ liệu theo thời gian**
- Không trả về một lần như Future
- Có thể phát:
  - nhiều giá trị
  - theo thời gian thực
- `StreamController`:
  - nơi **phát dữ liệu**
- `listen()`:
  - nơi **nhận & xử lý dữ liệu**

👉 Sinh viên phải phân biệt rõ:

- **Future = 1 lần**
- **Stream = nhiều lần**

🎯 Mục tiêu học tập (làm được gì trong app Flutter thật)

Sau phần này, sinh viên có thể:

- Xây dựng UI:
  - cập nhật liên tục
  - không cần reload thủ công
- Ứng dụng vào:
  - realtime data
  - notification
  - chat đơn giản
  - timer / counter

📘 Ý nghĩa sư phạm – vì sao cực kỳ quan trọng

- Stream là **trái tim của Flutter reactive**
- Là nền móng cho:
  - StreamBuilder
  - Bloc / Rx
  - Firebase realtime
- Nếu không hiểu Stream → **không hiểu kiến trúc Flutter nâng cao**

🔑 TỔNG KẾT TƯ DUY ASYNC CẦN KHẮC SÂU

- UI Flutter **luôn chạy trước**
- Dữ liệu async **luôn đến sau**
- Code không lỗi ≠ logic đúng
- Phải luôn tự hỏi:
  - "Dữ liệu này đã có chưa?"
  - "UI đang ở trạng thái nào?"

PhucPT10@fpt.edu.vn

📚 TÀI LIỆU THAM KHẢO UY TÍN (DÀNH CHO SINH VIÊN MỚI)

1. **Dart Official – Asynchronous Programming**
   - https://dart.dev/codelabs/async-await
2. **Flutter Docs – Asynchronous UI**
   - https://docs.flutter.dev/development/data-and-backend/async
3. **Flutter Codelabs – Fetch Data from Internet**
   - https://docs.flutter.dev/codelabs
4. **YouTube – Flutter Async for Beginners**
   - Kênh: *Flutter*, *Reso Coder*, *Code With Andrea*
5. **DartPad**
   - https://dartpad.dev
     👉 Thực hành async trực tiếp, không cần setup

🎯 KẾT QUẢ KỲ VỌNG SAU SLOT 4

Sau Slot 4, sinh viên:

- Hiểu **vì sao Flutter phải dùng async**
- Biết rõ:
  - khi nào dùng **Future**
  - khi nào dùng **Stream**
- Không còn:
  - sợ async
  - viết code "chạy nhưng sai"
- **Sẵn sàng bước sang:**
  - Networking
  - State Management
  - App Flutter thực tế quy mô lớn

PhucPT10@fpt.edu.vn

II. TRỌNG TÂM CỐT LÕI & KỸ NĂNG SINH VIÊN CẦN NẮM VỮNG

1 FUTURES & ASYNC BASICS – NỀN TẢNG TƯ DUY BẤT ĐỒNG BỘ

◈ Core Concepts – BẮT BUỘC PHẢI HIỂU

- **Async (bất đồng bộ) là gì?**
  - Code **không chờ kết quả xong mới chạy tiếp**
  - Dart **đẩy công việc nặng ra ngoài**, không làm đứng UI
- **Vì sao Flutter KHÔNG cho block UI?**
  - Flutter chạy UI trên **main thread**
  - Nếu chặn main thread → app **đơ, lag, bị kill**
- **Future là gì?**
  - Không phải dữ liệu
  - Là **đối tượng đại diện cho kết quả sẽ có trong tương lai**
- **Future chỉ có 2 trạng thái kết thúc**
  - `Completed with value`
  - `Completed with error`

👉 Sinh viên **bắt buộc phải hiểu**:
"Khi hàm trả về Future, dữ liệu CHƯA tồn tại tại thời điểm gọi"

◈ Kỹ năng bắt buộc phải THÀNH THẠO

- Nhận diện hàm:
  - đồng bộ (`String`, `int`)
  - bất đồng bộ (`Future<String>`)
- Gọi Future **không làm app đứng**
- Biết:
  - dữ liệu chưa có → phải chờ
- Kết hợp với Flutter UI:
  - hiển thị **loading**
  - không truy cập dữ liệu quá sớm

◈ Lỗi thường gặp của sinh viên mới

- ❌ Dùng dữ liệu Future như dữ liệu thường
  → **Nguyên nhân:** không hiểu "Future ≠ value"
- ❌ In ra `Instance of 'Future'`
  → **Nguyên nhân:** chưa chờ Future hoàn thành
- ❌ UI hiển thị rỗng / sai dữ liệu
  → **Nguyên nhân:** build UI trước khi dữ liệu sẵn sàng

◈ Best Practices – Thực hành chuẩn

- Luôn tự hỏi:
  - "Dữ liệu này có **ngay** hay **đến sau**?"
- Đặt tên hàm rõ ràng:
  - `fetchData()`, `loadUser()`
- Không xử lý logic async trong `build()` trực tiếp

📝 Ý nghĩa sư phạm

- Đây là **cửa ngõ bắt buộc** để hiểu Flutter
- Không hiểu Future → **mọi kiến thức sau đều sai gốc**
- Giúp sinh viên **thoát khỏi nỗi sợ async**

PhucPT10@fpt.edu.vn

2 ASYNC / AWAIT & FUTURE HANDLING – VIẾT ASYNC ĐÚNG LOGIC

◈ Core Concepts – BẮT BUỘC PHẢI HIỂU

- `async`:
  - đánh dấu hàm có thể dùng `await`
- `await`:
  - chờ **Future hoàn thành**
  - **không block UI**
- `then()`:
  - xử lý kết quả sau khi Future hoàn thành
- `try / catch`:
  - bắt lỗi async **bắt buộc phải có**

👉 Sinh viên phải hiểu rõ:
"await chỉ làm hàm đó chờ – UI vẫn chạy bình thường"

◈ Kỹ năng bắt buộc phải THÀNH THẠO

- Viết hàm:
  - `Future<T> functionName() async`
- Gọi API giả lập:
  - `Future.delayed`
- Xử lý lỗi:
  - `try / catch`
- Cập nhật UI sau async:
  - đúng thời điểm
  - đúng trạng thái

◈ Lỗi thường gặp của sinh viên mới

- ❌ Quên `await`
  → Code chạy nhưng **logic sai**
- ❌ Không bắt lỗi async
  → App crash **không rõ nguyên nhân**
- ❌ Gọi async trong `build()`
  → UI chạy lặp vô hạn

◈ Best Practices – Thực hành chuẩn

- Ưu tiên:
  - `async / await` thay vì `then()` dài dòng
- Mỗi async task:
  - có **loading**
  - có **error handling**
- Tách:
  - logic async
  - UI render

📝 Ý nghĩa sư phạm

- Giúp sinh viên:
  - đọc hiểu code dự án thật
  - không viết async "mù"
- Là nền móng cho:
  - Networking
  - Repository pattern

PhucPT10@fpt.edu.vn

- State Management

3 STREAMS & REALTIME DATA – DỮ LIỆU CHẢY THEO THỜI GIAN

MAIN THREAD – UI THREAD – ASYNC TRONG FLUTTER

1 THREAD LÀ GÌ?

- Thread = **luồng thực thi trong CPU**
- Mỗi thread chỉ làm **1 việc tại 1 thời điểm**
- App mobile **rất nhạy cảm** với thread

2 UI THREAD (MAIN THREAD) – LUỒNG QUAN TRỌNG NHẤT

📱 UI Thread chịu trách nhiệm:

- Vẽ giao diện
- Nhận tap, scroll, animation
- Chạy `build()` trong Flutter

👄 **QUY TẮC SỐNG CÒN**

❌ UI THREAD KHÔNG ĐƯỢC BỊ BLOCK

📉 UI Thread bị block →

- UI đứng
- App lag / ANR
- Người dùng tưởng app bị lỗi

3 WORK / BACKGROUND TASK

- Dùng cho:
  - gọi API
  - đọc file
  - chờ dữ liệu
- Không vẽ UI
- Flutter dùng async để **không làm bận UI thread**

4 ASYNC / AWAIT HIỂU ĐÚNG

⚠️ async / await:

- ❌ KHÔNG tạo thread mới
- ❌ KHÔNG chạy song song
- ✅ Chỉ giúp **UI thread không bị chặn**

📌 await:

- chỉ dừng **hàm hiện tại**
- không dừng app
- không dừng UI

5 EVENT LOOP (CỐT LÕI FLUTTER)

- Flutter dùng **Event Loop**
- async = xếp việc vào hàng đợi
- Khi xong → quay lại xử lý tiếp

👉 Nhờ vậy:

- UI luôn có thời gian vẽ
- App mượt

PhucPT10@fpt.edu.vn

6 FUTURE & STREAM – LIÊN HỆ THREAD

Future

- Việc **chạy 1 lần**
- Ví dụ: gọi API, load data

Stream

- Dữ liệu **phát nhiều lần**
- Ví dụ: counter, chat, realtime

Future & Stream:

- ❌ không phải thread
- ❌ không phải đa luồng

7 NHỮNG HIỂU SAI RẤT PHỔ BIẾN

❌ async = chạy song song
❌ Stream = đa luồng
❌ dùng async là app tự động mượt
👉 **Đều sai**

8 TƯ DUY ĐÚNG CẦN NHỚ (5 CÂU VÀNG)

1️⃣ UI Thread = vẽ giao diện
2️⃣ Block UI Thread = app chết
3️⃣ async giúp UI Thread rảnh
4️⃣ Future = bất đồng bộ 1 lần
5️⃣ Stream = dữ liệu theo thời gian

🎯 KẾT LUẬN CHỐT

Async trong Flutter không phải là cú pháp – mà là tư duy bảo vệ UI Thread
Hiểu được điều này →
✅ viết Future đúng
✅ dùng Stream đúng
✅ sẵn sàng học API, State Management, app lớn

Core Concepts – BẮT BUỘC PHẢI HIỂU

- **Stream là gì?**
  - Dòng dữ liệu **phát nhiều lần**
- Khác với Future:
  - Future → 1 kết quả
  - Stream → nhiều kết quả theo thời gian
- `StreamController`:
  - nơi **tạo & phát dữ liệu**
- `listen()`:
  - nơi **nhận dữ liệu**

👉 Sinh viên phải phân biệt:
"Future dùng cho lấy dữ liệu một lần – Stream cho realtime"

Kỹ năng bắt buộc phải THÀNH THẠO

- Tạo Stream đơn giản
- Dùng `StreamController`
- `listen()` để:
  - nhận dữ liệu

PhucPT10@fpt.edu.vn

    - xử lý từng lần cập nhật
- Kết nối Stream với UI:
    - realtime counter
    - timer
    - dữ liệu thay đổi liên tục

◈ Lỗi thường gặp của sinh viên mới

- ❌ Dùng Stream cho dữ liệu chỉ cần 1 lần
  → **Sai tư duy thiết kế**
- ❌ Quên đóng Stream (`close`)
  → Rò rỉ bộ nhớ
- ❌ Không hiểu dữ liệu đến nhiều lần
  → UI update sai

◈ Best Practices – Thực hành chuẩn

- Chỉ dùng Stream khi:
    - dữ liệu **thay đổi liên tục**
- Luôn:
    - quản lý vòng đời Stream
- Tách:
    - nguồn dữ liệu
    - UI hiển thị

📝 Ý nghĩa sư phạm

- Stream là **xương sống của Flutter reactive**
- Không hiểu Stream → **không học được Bloc, Firebase**
- Chuẩn bị tư duy cho:
    - Realtime app
    - State Management nâng cao

🎯 TỔNG KẾT TRỌNG TÂM BẮT BUỘC PHẢI ĐẠT

☑ Sinh viên BẮT BUỘC PHẢI HIỂU

- Async ≠ chạy song song
- Future ≠ dữ liệu
- UI Flutter **không được block**
- Code không lỗi ≠ logic đúng

☑ Sinh viên PHẢI LÀM ĐƯỢC

- Viết hàm async đúng
- Gọi API giả lập
- Hiển thị loading
- Xử lý lỗi async
- Phân biệt:
    - Future vs Stream

PhucPT10@fpt.edu.vn

III. ỨNG DỤNG THỰC TIỄN

🧠 ASYNC TRONG FLUTTER KHÔNG PHẢI CÚ PHÁP – MÀ LÀ TƯ DUY DỰ ÁN

Trước khi đi vào tình huống cụ thể, sinh viên **bắt buộc phải hiểu một sự thật**:
Flutter app = UI luôn chạy trước, dữ liệu luôn đến sau
→ Nếu không dùng async đúng, app **không đơ thì cũng sai logic**
Slot 4 không dạy "viết Future cho biết", mà dạy:

- **Giữ UI mượt**
- **Dữ liệu về đúng thời điểm**
- **Logic dễ debug – dễ mở rộng**

1 ỨNG DỤNG ASYNC TRONG PHÁT TRIỂN DỰ ÁN FLUTTER THỰC TẾ

◈ Vai trò của async trong app Flutter thật

Trong **100% app Flutter thực tế**, async được dùng để:

- ⏳ **Xử lý tác vụ mất thời gian**
  - gọi API
  - đọc file
  - truy vấn database
- 🎨 **Giữ UI không bị block**
  - vẫn scroll
  - vẫn click
  - vẫn animate
- 🔄 **Cập nhật dữ liệu động**
  - dữ liệu về sau
  - dữ liệu thay đổi liên tục (realtime)

👉 **Không có async → Flutter app không tồn tại**

◈ Slot 4 là nền tảng bắt buộc cho các slot sau

- Networking:
  - mọi API đều trả về `Future`
- State Management:
  - bản chất là **quản lý trạng thái async**
- App quy mô lớn:
  - càng lớn → async càng nhiều → càng dễ sai nếu hiểu mơ hồ

2 CÁC TÌNH HUỐNG THỰC TẾ ÁP DỤNG ASYNC (4 TÌNH HUỐNG)

◈ TÌNH HUỐNG 1: APP GỌI API HIỂN THỊ DANH SÁCH

📍 Bối cảnh thực tế

- App sinh viên / app nội bộ
- Màn hình:
  - mở lên
  - gọi API
  - hiển thị danh sách dữ liệu

❗ Vấn đề thường gặp (liên quan async)

- UI trắng / null
- In ra `Instance of Future`
- App không lỗi nhưng **không hiển thị gì**

👉 **Nguyên nhân gốc rễ**:

- Không hiểu:
  - API trả dữ liệu **sau**

PhucPT10@fpt.edu.vn

    - UI build **trước**

🛠 Cách áp dụng kỹ thuật Slot 4

- Dùng **Future** cho:
    - API gọi **1 lần**
- Dùng `async / await` để:
    - chờ dữ liệu
    - xử lý rõ ràng từng bước
- Hiển thị:
    - loading khi chưa có data
    - data khi Future hoàn thành
- Bắt lỗi bằng:
    - `try / catch`

🎯 Mục tiêu đạt được

- UI không bị block
- Dữ liệu hiển thị đúng thời điểm
- Logic dễ đọc – dễ debug
- Sinh viên hiểu:
  "Không phải code sai – mà là **chưa chờ dữ liệu**"

◈ TÌNH HUỐNG 2: MVP STARTUP – FORM SUBMIT & LOADING

📍 Bối cảnh thực tế

- MVP cho startup nhỏ
- Người dùng:
    - nhập form
    - bấm Submit
    - gửi dữ liệu lên server

❗ Vấn đề thường gặp

- Bấm submit nhiều lần
- Không biết gửi thành công hay chưa
- App bị "đơ" trong lúc gửi

👉 **Nguyên nhân**:

- Gọi API đồng bộ
- Không quản lý trạng thái async

🛠 Cách áp dụng kỹ thuật Slot 4

- Dùng `async / await` khi submit form
- Trước `await`:
    - bật loading
    - disable nút submit
- Sau `await`:
    - thành công → thông báo
    - lỗi → hiển thị error
- Dùng `catchError` / `try-catch` để:
    - không crash app

🎯 Mục tiêu đạt được

- UX tốt hơn 🚀
- Không submit trùng
- Trạng thái rõ ràng

PhucPT10@fpt.edu.vn

- Code dễ mở rộng (thêm validation, retry…)

◈ TÌNH HUỐNG 3: APP REALTIME – COUNTER / TIMER / NOTIFICATION

📍 Bối cảnh thực tế

- App demo realtime
- App nội bộ hiển thị:
  - số lượng online
  - timer
  - thông báo đơn giản

❗ Vấn đề thường gặp

- Dùng Future → dữ liệu chỉ cập nhật 1 lần
- UI không thay đổi theo thời gian
- Sinh viên "không hiểu vì sao data không update"

🛠 Cách áp dụng kỹ thuật Slot 4

- Dùng **Stream** khi:
  - dữ liệu thay đổi liên tục
- Dùng `StreamController`:
  - phát dữ liệu theo thời gian
- Dùng `listen()`:
  - nhận từng lần cập nhật
- Kết nối Stream với UI:
  - mỗi event → UI update

🎯 Mục tiêu đạt được

- UI cập nhật realtime
- Hiểu rõ:
  - Future ≠ Stream
- Chuẩn bị tư duy cho:
  - Firebase
  - Chat app
  - Notification system

◈ TÌNH HUỐNG 4: CODE CHẠY KHÔNG LỖI – NHƯNG SAI LOGIC

📍 Bối cảnh thực tế

- App sinh viên
- Code build được
- Không crash
- Nhưng dữ liệu hiển thị sai

❗ Vấn đề

- Quên `await`
- Logic chạy trước khi data về
- Không bắt lỗi async

👉 **Đây là lỗi nguy hiểm nhất trong dự án thật**

🛠 Cách áp dụng kỹ thuật Slot 4

- Luôn kiểm tra:
  - hàm có trả Future không?
- Bắt buộc:

PhucPT10@fpt.edu.vn

    - `await` khi cần data
- Tách:
    - async logic
    - UI logic
- Luôn xử lý error async

🎯 Mục tiêu đạt được

- Code **đúng logic**, không chỉ "chạy được"
- Debug dễ
- Dự án ổn định lâu dài

3 CASE STUDY – ỨNG DỤNG ASYNC THÀNH CÔNG

📍 Bối cảnh dự án

- App nội bộ hiển thị danh sách + realtime counter
- Dữ liệu:
    - load từ API
    - cập nhật theo thời gian

❗ Vấn đề ban đầu

- UI lag
- Code async lộn xộn
- Khó debug
- Dữ liệu về sai thứ tự

🛠 Áp dụng Slot 4

- **Future**
    - load data ban đầu
- **async / await**
    - xử lý tuần tự
- **Stream**
    - cập nhật realtime counter
- **Error handling**
    - tránh crash

✅ Kết quả

- UI mượt
- Logic rõ ràng
- Dữ liệu ổn định
- Dễ mở rộng sang State Management

4 BÀI HỌC RÚT RA – LESSONS LEARNED

🔑 Điều QUAN TRỌNG NHẤT

- Async là **tư duy**, không phải cú pháp
- Phải luôn hiểu:
    - dữ liệu đến **khi nào**
    - UI đang ở **trạng thái nào**

❌ Sai lầm sinh viên hay mắc

- Nghĩ async là chạy song song
- Dùng Future cho mọi thứ
- Lạm dụng Stream

PhucPT10@fpt.edu.vn

- Không xử lý lỗi async

✅ Nguyên tắc làm Flutter app lâu dài

- Không block UI
- Tách async logic khỏi UI
- Xử lý lỗi ngay từ đầu
- Luôn hiểu luồng chạy code

PhucPT10@fpt.edu.vn

IV. MỘT SỐ NỘI DUNG CHI TIẾT

PHẦN A. TƯ DUY BẤT ĐỒNG BỘ TRONG FLUTTER

I. ĐỒNG BỘ vs BẤT ĐỒNG BỘ (SYNC vs ASYNC)

◈ Code đồng bộ (Synchronous) là gì?

- Code **chạy từng dòng, theo thứ tự**
- Dòng sau **chỉ chạy khi dòng trước chạy xong**
- Nếu một dòng mất nhiều thời gian → **toàn bộ chương trình đứng chờ**

📌 **Ví dụ đời thực**

- Bạn đi photocopy:
  - đứng chờ in xong
  - xong mới làm việc khác

📉 **Vấn đề nếu dùng sync cho UI**

- UI bị đơ
- Không click được
- App bị kill

◈ Code bất đồng bộ (Asynchronous) là gì?

- Công việc mất thời gian:
  - được **đưa đi xử lý riêng**
- Chương trình **không đứng chờ**
- Khi xong → **báo kết quả quay lại**

📌 **Ví dụ đời thực**

- Gọi đồ ăn:
  - gọi xong → làm việc khác
  - khi xong → nhân viên báo

◈ Flutter UI Thread hoạt động như thế nào?

- Flutter có **UI thread (main thread)**
- Thread này chịu trách nhiệm:
  - vẽ UI
  - nhận sự kiện người dùng
- Nếu thread này bị block:
  - UI **đóng băng**

👉 **Flutter CẤM block UI thread**

◈ Sinh viên thường hiểu sai ở đâu?

- Nghĩ rằng:
  - code dài thì Flutter tự tối ưu
- Nghĩ async là:
  - "cho vui"
  - "cho đúng cú pháp"

❗ Hậu quả nếu hiểu sai async

- UI lag
- Dữ liệu sai
- App crash khó debug
- Code không mở rộng được

II. EVENT LOOP & LUỒNG THỰC THI ASYNC TRONG DART

◈ Event Loop là gì? (hiểu đơn giản)

- Dart chạy theo **vòng lặp sự kiện**

PhucPT10@fpt.edu.vn

- Mỗi lần:
  - lấy việc tiếp theo ra xử lý
- Không chạy song song trên 1 thread

👉 **Async ≠ đa luồng**

◈ Hai hàng đợi trong Dart (mức giới thiệu)

- **Microtask Queue**
  - ưu tiên cao
  - chạy trước
- **Event Queue**
  - Future, timer, IO

📌 Dart luôn:

1. Chạy code sync
2. Chạy microtask
3. Chạy event

◈ Vì sao `print()` không chạy theo thứ tự sinh viên nghĩ?

Ví dụ logic:

- Code sync chạy trước
- Future chạy sau
- Nhưng sinh viên nghĩ:
  - "viết trước thì chạy trước"

👉 **Sai do không hiểu event loop**

❗ Vì sao code async "chạy nhưng sai logic"?

- Không chờ dữ liệu
- Không hiểu:
  - đoạn nào chạy ngay
  - đoạn nào chạy sau

PHẦN B. FUTURE & ASYNC / AWAIT – NỀN TẢNG BẮT BUỘC

III. FUTURE – KHÁI NIỆM CỐT LÕI

◈ Future là gì?

- Là **đối tượng đại diện cho kết quả trong tương lai**
- **KHÔNG phải dữ liệu**

👉 So sánh:

- `String name` → có ngay
- `Future<String> name` → chưa có

◈ Trạng thái của Future

- `pending`: đang chờ
- `completed with value`
- `completed with error`

📌 **Future chỉ kết thúc 1 lần**

◈ Future đại diện cho điều gì trong app Flutter?

- Gọi API
- Đọc file
- Query database
- Gửi form

PhucPT10@fpt.edu.vn

👉 **Mọi thứ mất thời gian đều là Future**

IV. ASYNC / AWAIT – VIẾT ASYNC CODE DỄ HIỂU

◈ Từ `then()` → `async / await`

- `then()`:
  - khó đọc
  - dễ callback hell
- `async / await`:
  - viết như code thường
  - dễ suy luận

◈ Vì sao `await` không block UI?

- `await`:
  - chỉ dừng **hàm đó**
- UI thread:
  - vẫn chạy bình thường

👉 **await ≠ sleep**

◈ Quy tắc BẮT BUỘC

- Hàm dùng `await` → phải có `async`
- Gọi Future mà cần kết quả → phải `await`

◈ Lỗi sinh viên hay gặp

- Quên `await`
- Nghĩ code chạy từ trên xuống
- Không hiểu dữ liệu chưa về

👉 **Đây là nguồn gốc của bug nguy hiểm**

V. XỬ LÝ LỖI ASYNC (ERROR HANDLING)

◈ Lỗi async khác lỗi sync thế nào?

- Lỗi sync:
  - xảy ra ngay
- Lỗi async:
  - xảy ra sau
  - dễ bị "nuốt mất"

◈ try / catch với async / await

- Bắt lỗi **bắt buộc**
- Nếu không:
  - app crash
  - khó debug

◈ catchError với Future

- Bắt lỗi theo kiểu chaining
- Khó đọc hơn async/await

❗ Vì sao app crash mà không báo rõ lỗi?

- Không xử lý error async
- Lỗi xảy ra ngoài luồng sync

PhucPT10@fpt.edu.vn

PHẦN C. STREAMS & DỮ LIỆU THỜI GIAN THỰC

🧠 HIỂU ĐÚNG: MAIN THREAD – UI THREAD – WORK THREAD
(Dành cho Flutter & Mobile Developer)

1 VÌ SAO PHẢI HỌC VỀ THREAD?

👄 Sinh viên hay gặp hiện tượng:

- App **đơ**
- UI **không bấm được**
- Loading **không xoay**
- Async viết đúng cú pháp nhưng **trải nghiệm tệ**

👉 **Nguyên nhân gốc**:
❌ Không hiểu **thread của hệ điều hành mobile**

2 KHÁI NIỆM CỐT LÕI (KHÔNG ĐƯỢC LƯỚT)

◈ THREAD LÀ GÌ?

Thread = một "luồng thực thi" trong CPU

📌 Ví dụ đời thường:

- Một người chỉ làm **1 việc tại 1 thời điểm**
- Muốn làm nhiều việc → phải **chia thời gian**

CPU cũng vậy.

3 MAIN THREAD / UI THREAD (CỰC KỲ QUAN TRỌNG)

◍ MAIN THREAD (hay UI THREAD)

👉 Là **luồng quan trọng nhất** trong app mobile
📱 Trách nhiệm của UI Thread:

- Vẽ UI
- Nhận sự kiện:
  - tap
  - scroll
  - animation
- Chạy hàm `build()` trong Flutter

👄 **QUY TẮC SỐNG CÒN**
❌ UI THREAD TUYỆT ĐỐI KHÔNG ĐƯỢC BỊ CHẶN (BLOCK)

📉 CHUYỆN GÌ XẢY RA NẾU UI THREAD BỊ BLOCK?

Hiện tượng	Sinh viên thường nghĩ
UI đứng	Flutter lỗi
App lag	Máy yếu
Loading không xoay	Widget sai

👉 **SỰ THẬT**:
UI Thread đang bận → không vẽ được

👄 Android gọi là:

- **ANR** (Application Not Responding)

👄 iOS:

- App bị kill

PhucPT10@fpt.edu.vn

4 WORK THREAD / BACKGROUND THREAD

WORK THREAD LÀ GÌ?

👉 Là thread **không vẽ UI**

👉 Dùng để:

- Gọi API
- Đọc file
- Tính toán nặng
- Delay / chờ

📌 Flutter:

- async task
- Future
- Stream

  👉 **đều nhằm tránh block UI thread**

5 FLUTTER DÙNG THREAD NHƯ THẾ NÀO?

⚠ SỰ THẬT QUAN TRỌNG (SV RẤT HAY HIỂU SAI)

❌ async / await **KHÔNG TẠO THREAD MỚI**

Flutter thực tế dùng:

- **Event Loop**
- **Single-threaded UI**
- Async = **chia thời gian thông minh**

6 MINH HỌA LUỒNG CHẠY (CỰC QUAN TRỌNG)

◈ CODE SYNC (NGUY HIỂM)

```
void main() {
  print('Start');

  sleep(Duration(seconds: 5)); // BLOCK UI

  print('End');
}
```

👄 KẾT QUẢ:

- App đứng 5 giây
- UI không vẽ
- Không bấm được

◈ CODE ASYNC (ĐÚNG)

```
Future<void> main() async {
  print('Start');

  await Future.delayed(Duration(seconds: 5));

  print('End');
}
```

🧠 GIẢI THÍCH:

- UI thread **không bị chặn**
- Event loop nhớ:
  "5 giây sau quay lại"

7 EVENT LOOP (HIỂU MỨC ĐỘ MOBILE)

📦 Flutter có:

PhucPT10@fpt.edu.vn

- **Event Queue**
- **Microtask Queue**

👉 async = "xếp việc vào hàng đợi"

📌 Khi UI rảnh:

- Flutter lấy task tiếp theo
- Không task nặng → UI mượt

8 STREAM LIÊN QUAN GÌ TỚI THREAD?

🔁 STREAM KHÔNG PHẢI THREAD

Stream = **dòng dữ liệu theo thời gian**

📌 Stream:

- phát dữ liệu
- UI nghe

👄 Stream **KHÔNG chạy song song**

👉 Nó **phát từng sự kiện**, UI xử lý từng cái

9 TƯ DUY SAI PHỔ BIẾN (BẮT BUỘC CHỈ RA)

❌ Tư duy 1

async = chạy song song

❌ SAI

✓ async = **không block UI**

❌ Tư duy 2

dùng async là app mượt

❌ SAI

✓ async sai chỗ → app vẫn lag

❌ Tư duy 3

Stream là đa luồng

❌ SAI

✓ Stream là **chuỗi sự kiện**

10 TƯ DUY ĐÚNG CHO SINH VIÊN FLUTTER

👉 **NHỚ 5 CÂU NÀY**

1️⃣ UI Thread = vẽ giao diện

2️⃣ UI Thread bị block = app chết

3️⃣ async giúp UI Thread rảnh

4️⃣ Future = việc 1 lần

5️⃣ Stream = việc lặp theo thời gian

🎓 LIÊN HỆ VỚI BÀI TẬP ĐÃ LÀM

📌 Trong bài Future Product Loader

- `Future.delayed` chạy nền
- UI vẫn hiện loading

📌 Trong bài Stream Counter

- Stream phát số
- UI update liên tục
- UI thread **không bị block**

PhucPT10@fpt.edu.vn

🧩 CÂU HỎI KIỂM TRA HIỂU (BẮT BUỘC)

1️⃣ Vì sao **sleep()** nguy hiểm trong Flutter?
2️⃣ await có tạo thread mới không?
3️⃣ Vì sao UI vẫn mượt khi chờ Future?
👉 Trả lời đúng = **HIỂU OS MOBILE**

VI. STREAM – TƯ DUY DỮ LIỆU NHIỀU LẦN

◈ Stream là gì?

- Dòng dữ liệu phát ra **nhiều lần theo thời gian**

👉 So sánh:

- Future → 1 lần
- Stream → nhiều lần

◈ Khi nào KHÔNG nên dùng Stream?

- Chỉ cần data 1 lần
- Data không thay đổi

👉 **Dùng Stream sai → code phức tạp**

◈ Ví dụ thực tế

- Chat
- Notification
- Realtime counter
- Firebase

VII. STREAMCONTROLLER

◈ StreamController là gì?

- Nơi:
  - tạo Stream
  - phát dữ liệu

◈ Các thao tác cơ bản

- `add()` → phát data
- `listen()` → nhận data
- `close()` → giải phóng

◈ Single vs Broadcast (mức giới thiệu)

- Single:
  - 1 listener
- Broadcast:
  - nhiều listener

❗ Tránh memory leak

- Không đóng Stream
- UI dispose nhưng Stream còn chạy

VIII. STREAM TRONG FLUTTER UI (MỨC NỀN TẢNG)

◈ Vì sao Stream hợp với UI realtime?

- UI Flutter là reactive
- Stream phát data → UI cập nhật

PhucPT10@fpt.edu.vn

◈ Lỗi thường gặp

- Lạm dụng Stream
- Không quản lý vòng đời
- UI update sai

PHẦN D. LỖI PHỔ BIẾN & TƯ DUY ĐÚNG

IX. NHỮNG SAI LẦM KINH ĐIỂN

✕ Nghĩ async là chạy song song

- Nguyên nhân: nhầm async = multi-thread
- Sửa: hiểu event loop

✕ Dùng async nhưng vẫn block UI

- Nguyên nhân: xử lý nặng trong main thread
- Sửa: chỉ async IO, không heavy compute

✕ Lạm dụng Stream

- Nguyên nhân: chưa phân biệt Future vs Stream
- Sửa: hỏi "data có thay đổi liên tục không?"

✕ Không xử lý lỗi async

- Nguyên nhân: học cú pháp, bỏ tư duy
- Sửa: luôn có error handling

X. TỔNG KẾT & ĐỊNH HƯỚNG

🔑 Vai trò của async trong Flutter

- Async = **xương sống của Flutter app**
- Không hiểu async → không làm app thật

PhucPT10@fpt.edu.vn

V. KINH NGHIỆM THỰC TIỄN, TIPS

🧠 ASYNC TRONG DỰ ÁN THẬT

Async trong sách = ví dụ đẹp
Async ngoài đời = dữ liệu đến muộn, lỗi khó debug, UI dễ sai
Nếu sinh viên chỉ học async qua ví dụ đơn giản → **100% sẽ vấp lỗi khi làm project**.
Phần này giúp **đi trước lỗi 1 bước**.

1️⃣ KỸ THUẬT: FUTURE

✅ Kinh nghiệm thực tế khi dùng Future

📍 Dùng Future ở đâu trong app thật?

- Gọi API **1 lần**
- Load dữ liệu ban đầu (profile, list)
- Submit form
- Đọc file / local storage

👉 **Future = tác vụ "đến một lần rồi thôi"**

📌 Điều KHÔNG giống trong sách

- API ngoài đời:
  - chậm
  - lỗi
  - timeout
- Data:
  - có thể `null`
  - có thể về trễ hơn bạn nghĩ

⚠️ LỖI PHỔ BIẾN VỚI FUTURE

❌ LỖI 1: "Code chạy nhưng dữ liệu sai / null"

Biểu hiện

- App không crash
- UI không hiển thị data
- Log không báo lỗi

Nguyên nhân gốc

- Quên `await`
- Hiểu sai: *"viết trước thì chạy trước"*

🧪 **Ví dụ tình huống**

- Gọi API
- Gán data cho biến
- UI dùng biến đó ngay

👉 Nhưng API **chưa trả kết quả**

🛠 **Cách khắc phục**

- Luôn `await` khi cần kết quả
- Không dùng data Future trước khi hoàn thành

🚫 **Phòng tránh**

- Quy tắc cá nhân:
  "Hàm trả Future → phải kiểm tra đã await chưa"

❌ LỖI 2: Dùng Future cho dữ liệu thay đổi liên tục

Biểu hiện

- UI chỉ update 1 lần

PhucPT10@fpt.edu.vn

- Phải reload màn hình

Nguyên nhân

- Dùng Future cho bài toán realtime

🛠 **Cách sửa**

- Chuyển sang Stream

💡 Tips & Tricks với Future

- Đặt tên hàm rõ nghĩa:
  - `fetchUser()`, `loadProducts()`
- Khi debug:
  - log **trước await**
  - log **sau await**
    → thấy rõ thứ tự chạy

🛡 Best Practices – Future

- Chỉ dùng Future khi:
  - dữ liệu **không cần realtime**
- Mỗi Future:
  - có xử lý lỗi
- Không gọi Future lặp trong UI lifecycle

🎓 Bài học rút ra

- **Future ≠ dữ liệu**
- Quên await = bug nguy hiểm nhất
- Nếu chỉ nhớ 1 câu:
  "Future chỉ hứa – chưa giao hàng"

2 KỸ THUẬT: ASYNC / AWAIT

✅ Kinh nghiệm thực tế khi dùng async/await

🔑 Dùng async/await ở đâu?

- API call
- Submit form
- Chuỗi logic phụ thuộc dữ liệu

👉 async/await giúp **đọc code như câu chuyện**

📌 Điều không giống trong ví dụ đơn giản

- Async thật:
  - có loading
  - có disable UI
  - có retry
  - có error nhiều kiểu

⚠ LỖI PHỔ BIẾN

❌ LỖI 1: Quên await – lỗi "sát thủ"

Biểu hiện

- App không crash
- Logic sai ngầm
- Debug cực khó

Nguyên nhân

PhucPT10@fpt.edu.vn

- Tư duy sync
- Nghĩ async "tự chờ"

🛠 **Cách khắc phục**

- Code review: **scan await**
- IDE warning là bạn thân

❌ LỖI 2: await nhưng vẫn block UI

Biểu hiện

- UI lag
- Scroll giật

Nguyên nhân

- Xử lý nặng trong async nhưng vẫn ở main thread

🛠 **Cách sửa**

- Async chỉ dành cho IO
- Không xử lý tính toán nặng trong async UI

💡 Tips & Tricks

- Mỗi hàm async:
  - chỉ làm **1 nhiệm vụ**
- Đặt tên:
  - `async` → hành động
- Log:
  - đầu hàm
  - cuối hàm

🛡 Best Practices – async/await

- Ưu tiên async/await hơn then()
- Không lồng async quá sâu
- Tách async logic khỏi UI

🎓 Bài học rút ra

- await **không làm app đứng**
- async giúp code **đúng logic**
- Nhớ 1 điều:
  "await dừng hàm – không dừng UI"

3 KỸ THUẬT: XỬ LÝ LỖI ASYNC

✅ Kinh nghiệm thực tế

🔑 Lỗi async trong dự án thật

- Network fail
- Server error
- Timeout
- Data sai format

👉 **Async không lỗi ≠ app an toàn**

⚠ LỖI PHỔ BIẾN

❌ LỖI: Không bắt lỗi async

Biểu hiện

- App crash ngẫu nhiên

PhucPT10@fpt.edu.vn

- Không trace được lỗi

Nguyên nhân

- Không try/catch
- Nghĩ async tự handle

🛠 **Cách sửa**

- Mọi async call:
    - phải có error handling

💡 Tips

- Log lỗi async ngay khi catch
- Hiển thị thông báo cho user
- Không im lặng nuốt lỗi

🛡 Best Practices

- try/catch cho async/await
- catchError cho then
- Không để Future lỗi "rơi tự do"

🎓 Bài học rút ra

- Lỗi async **đến muộn**
- Không bắt → crash khó hiểu
- Nhớ:
  "Async không có error handling = app không kiểm soát"

4 KỸ THUẬT: STREAM

✅ Kinh nghiệm thực tế khi dùng Stream

🔑 Dùng Stream khi nào?

- Realtime counter
- Chat
- Notification
- Firebase

👉 **Stream = dữ liệu thay đổi liên tục**

📌 Điều sinh viên hay làm sai

- Dùng Stream cho API load 1 lần
- Khiến code phức tạp không cần thiết

⚠ LỖI PHỔ BIẾN

❌ LỖI: Lạm dụng Stream

Biểu hiện

- Code khó đọc
- Khó debug

Nguyên nhân

- Không phân biệt Future vs Stream

🛠 **Cách sửa**

- Hỏi 1 câu:
  "Data có thay đổi nhiều lần không?"

PhucPT10@fpt.edu.vn

❌ LỖI: Quên đóng Stream

Biểu hiện

- Memory leak
- UI dispose nhưng data vẫn chạy

🛠 Cách sửa

- Luôn `close()` StreamController

💡 Tips & Tricks

- Tên Stream:
  - thể hiện realtime (`counterStream`)
- Log mỗi event để debug

🛡 Best Practices – Stream

- Chỉ dùng khi thật sự realtime
- Quản lý vòng đời
- Không dùng Stream cho mọi thứ

🎓 Bài học rút ra

- Stream rất mạnh
- Dùng sai → hại hơn không dùng
- Nhớ:
  "Realtime mới dùng Stream"

5 KỸ THUẬT: STREAMCONTROLLER

⚠ LỖI KINH ĐIỂN

❌ Không biết ai tạo – ai nghe

Biểu hiện

- Data không đến UI
- Không hiểu luồng

Nguyên nhân

- Không rõ vai trò:
  - producer
  - consumer

🛠 Cách sửa

- Rõ ràng:
  - ai add()
  - ai listen()

🛡 Best Practices

- Controller tạo ở logic layer
- UI chỉ listen
- Close đúng vòng đời

🎓 Bài học rút ra

- StreamController = nguồn phát
- UI = người nghe
- Không lẫn vai trò

PhucPT10@fpt.edu.vn

TỔNG KẾT KINH NGHIỆM SLOT 4

Điều QUAN TRỌNG NHẤT

- Async = tư duy
- Không hiểu luồng → bug ngầm
- UI mượt quan trọng hơn "code chạy"

Cảnh báo tư duy sai

- async ≠ chạy song song
- dùng async sai còn hại hơn sync
- Stream không phải thuốc chữa bách bệnh

Nếu sinh viên chỉ nhớ 3 điều

1. Future ≠ data
2. Quên await = bug nặng
3. Realtime mới dùng Stream

ASYNC TRONG DỰ ÁN THẬT

Async trong sách = ví dụ đẹp
Async ngoài đời = dữ liệu đến muộn, lỗi khó debug, UI dễ sai
Nếu sinh viên chỉ học async qua ví dụ đơn giản → **100% sẽ vấp lỗi khi làm project**.
Phần này giúp **đi trước lỗi 1 bước**.

1 KỸ THUẬT: FUTURE

Kinh nghiệm thực tế khi dùng Future

Dùng Future ở đâu trong app thật?

- Gọi API **1 lần**
- Load dữ liệu ban đầu (profile, list)
- Submit form
- Đọc file / local storage

👉 **Future = tác vụ "đến một lần rồi thôi"**

Điều KHÔNG giống trong sách

- API ngoài đời:
  - chậm
  - lỗi
  - timeout
- Data:
  - có thể `null`
  - có thể về trễ hơn bạn nghĩ

⚠ LỖI PHỔ BIẾN VỚI FUTURE

❌ LỖI 1: "Code chạy nhưng dữ liệu sai / null"

Biểu hiện

- App không crash
- UI không hiển thị data
- Log không báo lỗi

Nguyên nhân gốc

- Quên `await`
- Hiểu sai: *"viết trước thì chạy trước"*

🧪 **Ví dụ tình huống**

PhucPT10@fpt.edu.vn

- Gọi API
- Gán data cho biến
- UI dùng biến đó ngay

👉 Nhưng API **chưa trả kết quả**

🛠 **Cách khắc phục**

- Luôn `await` khi cần kết quả
- Không dùng data Future trước khi hoàn thành

🚫 **Phòng tránh**

- Quy tắc cá nhân:
  "Hàm trả Future → phải kiểm tra đã await chưa"

❌ LỖI 2: Dùng Future cho dữ liệu thay đổi liên tục

Biểu hiện

- UI chỉ update 1 lần
- Phải reload màn hình

Nguyên nhân

- Dùng Future cho bài toán realtime

🛠 **Cách sửa**

- Chuyển sang Stream

💡 Tips & Tricks với Future

- Đặt tên hàm rõ nghĩa:
  - `fetchUser()`, `loadProducts()`
- Khi debug:
  - log **trước await**
  - log **sau await**
    → thấy rõ thứ tự chạy

🛡 Best Practices – Future

- Chỉ dùng Future khi:
  - dữ liệu **không cần realtime**
- Mỗi Future:
  - có xử lý lỗi
- Không gọi Future lặp trong UI lifecycle

🎓 Bài học rút ra

- **Future ≠ dữ liệu**
- Quên await = bug nguy hiểm nhất
- Nếu chỉ nhớ 1 câu:
  "Future chỉ hứa – chưa giao hàng"

2 KỸ THUẬT: ASYNC / AWAIT

✅ Kinh nghiệm thực tế khi dùng async/await

🔑 Dùng async/await ở đâu?

- API call
- Submit form
- Chuỗi logic phụ thuộc dữ liệu

👉 async/await giúp **đọc code như câu chuyện**

PhucPT10@fpt.edu.vn

📌 Điều không giống trong ví dụ đơn giản

- Async thật:
  - có loading
  - có disable UI
  - có retry
  - có error nhiều kiểu

⚠ LỖI PHỔ BIẾN

❌ LỖI 1: Quên await – lỗi "sát thủ"

Biểu hiện

- App không crash
- Logic sai ngầm
- Debug cực khó

Nguyên nhân

- Tư duy sync
- Nghĩ async "tự chờ"

🛠 Cách khắc phục

- Code review: **scan await**
- IDE warning là bạn thân

❌ LỖI 2: await nhưng vẫn block UI

Biểu hiện

- UI lag
- Scroll giật

Nguyên nhân

- Xử lý nặng trong async nhưng vẫn ở main thread

🛠 Cách sửa

- Async chỉ dành cho IO
- Không xử lý tính toán nặng trong async UI

💡 Tips & Tricks

- Mỗi hàm async:
  - chỉ làm **1 nhiệm vụ**
- Đặt tên:
  - `async` → hành động
- Log:
  - đầu hàm
  - cuối hàm

🛡 Best Practices – async/await

- Ưu tiên async/await hơn then()
- Không lồng async quá sâu
- Tách async logic khỏi UI

🎓 Bài học rút ra

- await **không làm app đứng**
- async giúp code **đúng logic**
- Nhớ 1 điều:
  "await dừng hàm – không dừng UI"

PhucPT10@fpt.edu.vn

3 KỸ THUẬT: XỬ LÝ LỖI ASYNC

✅ Kinh nghiệm thực tế

📍 Lỗi async trong dự án thật

- Network fail
- Server error
- Timeout
- Data sai format

👉 **Async không lỗi ≠ app an toàn**

⚠ LỖI PHỔ BIẾN

❌ LỖI: Không bắt lỗi async

Biểu hiện

- App crash ngẫu nhiên
- Không trace được lỗi

Nguyên nhân

- Không try/catch
- Nghĩ async tự handle

🛠 **Cách sửa**

- Mọi async call:
  - phải có error handling

💡 Tips

- Log lỗi async ngay khi catch
- Hiển thị thông báo cho user
- Không im lặng nuốt lỗi

🛡 Best Practices

- try/catch cho async/await
- catchError cho then
- Không để Future lỗi "rơi tự do"

🎓 Bài học rút ra

- Lỗi async **đến muộn**
- Không bắt → crash khó hiểu
- Nhớ:
  "Async không có error handling = app không kiểm soát"

4 KỸ THUẬT: STREAM

✅ Kinh nghiệm thực tế khi dùng Stream

📍 Dùng Stream khi nào?

- Realtime counter
- Chat
- Notification
- Firebase

👉 **Stream = dữ liệu thay đổi liên tục**

📌 Điều sinh viên hay làm sai

- Dùng Stream cho API load 1 lần

PhucPT10@fpt.edu.vn

- Khiến code phức tạp không cần thiết

⚠ LỖI PHỔ BIẾN

❌ LỖI: Lạm dụng Stream

Biểu hiện

- Code khó đọc
- Khó debug

Nguyên nhân

- Không phân biệt Future vs Stream

🛠 Cách sửa

- Hỏi 1 câu:
  "Data có thay đổi nhiều lần không?"

❌ LỖI: Quên đóng Stream

Biểu hiện

- Memory leak
- UI dispose nhưng data vẫn chạy

🛠 Cách sửa

- Luôn `close()` StreamController

💡 Tips & Tricks

- Tên Stream:
  - thể hiện realtime (`counterStream`)
- Log mỗi event để debug

🛡 Best Practices – Stream

- Chỉ dùng khi thật sự realtime
- Quản lý vòng đời
- Không dùng Stream cho mọi thứ

🎓 Bài học rút ra

- Stream rất mạnh
- Dùng sai → hại hơn không dùng
- Nhớ:
  "Realtime mới dùng Stream"

5 KỸ THUẬT: STREAMCONTROLLER

⚠ LỖI KINH ĐIỂN

❌ Không biết ai tạo – ai nghe

Biểu hiện

- Data không đến UI
- Không hiểu luồng

Nguyên nhân

- Không rõ vai trò:
  - producer
  - consumer

🛠 Cách sửa

- Rõ ràng:
  - ai add()

PhucPT10@fpt.edu.vn

- ai listen()

🛡 Best Practices

- Controller tạo ở logic layer
- UI chỉ listen
- Close đúng vòng đời

🎓 Bài học rút ra

- StreamController = nguồn phát
- UI = người nghe
- Không lẫn vai trò

🔚 TỔNG KẾT KINH NGHIỆM SLOT 4

🔑 Điều QUAN TRỌNG NHẤT

- Async = tư duy
- Không hiểu luồng → bug ngầm
- UI mượt quan trọng hơn "code chạy"

🚨 Cảnh báo tư duy sai

- async ≠ chạy song song
- dùng async sai còn hại hơn sync
- Stream không phải thuốc chữa bách bệnh

🎯 Nếu sinh viên chỉ nhớ 3 điều

4. Future ≠ data
5. Quên await = bug nặng
6. Realtime mới dùng Stream

PhucPT10@fpt.edu.vn

EXERCISE

BÀI TẬP TỔNG HỢP 1

BƯỚC 1: TẠO PROJECT & LÀM SẠCH main.dart

Mục tiêu

- Có **1 app Flutter chạy được**
- **Không async – không Future** ở bước này
- Sinh viên **hiểu app Flutter chạy từ đâu**

THAO TÁC CHI TIẾT (VS CODE)

1 Mở VS Code

2 Mở Terminal trong VS Code

```
Ctrl + `
```

3 Tạo project Flutter

```
flutter create async_product_loader
```

Chờ chạy xong (1–2 phút)

4 Mở thư mục project

```
File → Open Folder → async_product_loader
```

MỞ FILE: `lib/main.dart`

XÓA TOÀN BỘ CODE CŨ

DÁN CODE SAU (TỐI GIẢN)

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      home: Scaffold(
        body: Center(
          child: Text('Async Product Loader'),
        ),
      ),
    );
  }
}
```

KIỂM TRA

```
flutter run
```

App hiện chữ **Async Product Loader**

GIẢI THÍCH CHO SV

- App Flutter **chạy từ `main()`**
- `runApp()` gắn UI vào màn hình
- **CHƯA CÓ async – CHƯA CÓ Future**

PhucPT10@fpt.edu.vn

◈ BƯỚC 2: TẠO MODEL DỮ LIỆU (Item)

🎯 Mục tiêu

- Tách **dữ liệu** khỏi UI
- Chuẩn bị dữ liệu cho Future trả về
- KHÔNG hard-code trong Widget

🛠 THAO TÁC

1 Chuột phải thư mục `lib`

```
New Folder → models
```

2 Chuột phải `models`

```
New File → item.dart
```

📋 CODE: `lib/models/item.dart`

```
class Item {
  final String name;
  final double price;

  Item({
    required this.name,
    required this.price,
  });
}
```

🧠 GIẢI THÍCH (NÓI RÕ CHO SV MỚI)

- `Item` = **1 sản phẩm**
- `final` = không thay đổi sau khi tạo
- Model **KHÔNG chứa UI**
- Model **KHÔNG async**

◈ BƯỚC 3: TẠO FUTURE GIẢ LẬP (Fake API Service)

🎯 Mục tiêu

- TẠO **Future<List<Item>>**
- Hiểu **Future chạy chậm**
- Giả lập server bằng `Future.delayed`

🛠 THAO TÁC

1 Chuột phải `lib`

```
New Folder → services
```

2 Chuột phải `services`

```
New File → fake_api_service.dart
```

📋 CODE: `lib/services/fake_api_service.dart`

```
import '../models/item.dart';

class FakeApiService {
  static Future<List<Item>> fetchItems() async {
    print('🟡 1. Bắt đầu gọi fake API');

    await Future.delayed(const Duration(seconds: 2));
```


PhucPT10@fpt.edu.vn

```
    print('🟢 2. Fake API trả dữ liệu');

    return [
      Item(name: 'Táo', price: 10000),
      Item(name: 'Cam', price: 15000),
      Item(name: 'Chuối', price: 8000),
    ];
  }
}
```

🧠 GIẢI THÍCH CỰC CHẬM

Thành phần	Ý nghĩa
`Future<List<Item>>`	Không trả kết quả ngay
`async`	Trong hàm có việc chờ
`await`	Chỉ dừng hàm này
`Future.delayed`	Giả lập gọi server

👄 **NHẤN MẠNH**
`await` **KHÔNG làm đứng app**

🔷 BƯỚC 4: TEST FUTURE BẰNG print()

🎯 Mục tiêu

- Thấy **thứ tự chạy**
- Hiểu **await ảnh hưởng luồng code**
- Không dùng UI

🛠 THAO TÁC

1️⃣ MỞ FILE
`lib/services/fake_api_service.dart`

2️⃣ ĐỌC KỸ 2 DÒNG print()

```
print('🟡 1. Bắt đầu gọi fake API');
await Future.delayed(const Duration(seconds: 2));
print('🟢 2. Fake API trả dữ liệu');
```

▶️ CHẠY APP
`flutter run`

🔍 MỞ DEBUG CONSOLE

📌 KẾT QUẢ:

```
🟡 1. Bắt đầu gọi fake API
(sau 2 giây)
🟢 2. Fake API trả dữ liệu
```

🧠 THỰC NGHIỆM BẮT BUỘC

❌ BỎ `await`

```
Future.delayed(const Duration(seconds: 2));
```

▶️ CHẠY LẠI

📌 KẾT QUẢ:

```
🟡 1. Bắt đầu gọi fake API
🟢 2. Fake API trả dữ liệu
```

👄 **KẾT LUẬN**

PhucPT10@fpt.edu.vn

Không `await` → code chạy **SAI THỨ TỰ**

NHẬN DIỆN CHI TIẾT TỪNG BƯỚC

◈ BƯỚC 4.1: NHẬN DIỆN HÀM FUTURE

📌 Giải thích

Trong file `fake_api_service.dart`, tìm đoạn:

```
static Future<List<Item>> fetchItems() async {
```

Giải thích:

- `Future<List<Item>>`
  👉 "Hàm này **KHÔNG trả dữ liệu ngay**"
- `async`
  👉 "Trong hàm này có việc chạy mất thời gian"

◈ BƯỚC 4.2: THÊM print() THEO ĐÚNG THỨ TỰ

📋 Viết lại hàm fetchItems() NHƯ SAU

```
static Future<List<Item>> fetchItems() async {
  print('🟡 1. BẮT ĐẦU gọi fetchItems');

  await Future.delayed(const Duration(seconds: 2));

  print('🟢 2. SAU khi Future.delayed xong');

  return [
    Item(name: 'Táo', price: 10000),
    Item(name: 'Cam', price: 15000),
    Item(name: 'Chuối', price: 8000),
  ];
}
```

🧠 GIẢI THÍCH TỪNG DÒNG (KHÔNG ĐƯỢC LƯỚT)

◈ Dòng 1

```
print('🟡 1. BẮT ĐẦU gọi fetchItems');
```

👉 Dòng này chạy **NGAY LẬP TỨC**

◈ Dòng 2

```
await Future.delayed(const Duration(seconds: 2));
```

👉 Giải thích RẤT CHẬM:

- `Future.delayed`:
  - giả lập gọi server
  - mất 2 giây
- `await`:
  - **chỉ dừng HÀM NÀY**
  - **KHÔNG dừng app**
  - **KHÔNG dừng UI**

📌 Nói rõ:

"Nếu await mà block UI → Flutter không dùng được"

◈ Dòng 3

```
print('🟢 2. SAU khi Future.delayed xong');
```

👉 Chỉ chạy **SAU 2 GIÂY**

PhucPT10@fpt.edu.vn

BƯỚC 4.3: KHI CHẠY APP & MỞ CONSOLE

Thao tác

1. Mở Terminal
2. Chạy:
3. `flutter run`
4. Mở **Debug Console** (VS Code / Android Studio)

QUAN SÁT KẾT QUẢ (RẤT QUAN TRỌNG)

Console hiển thị:

```
🟡 1. BẮT ĐẦU gọi fetchItems
(sau 2 giây)
🟢 2. SAU khi Future.delayed xong
```

Nhận diện vấn đề

1 Vì sao dòng 🟢 không chạy ngay?

👉 Vì nó nằm **sau await**

2 Trong 2 giây chờ đó, app có bị đứng không?

👉 KHÔNG

3 Nếu bỏ `await` thì chuyện gì xảy ra?

BƯỚC 4.4: THỬ BỎ await (THỰC NGHIỆM)

❌ SỬA CODE (CỐ Ý SAI)

```
Future.delayed(const Duration(seconds: 2));
```

(không có await)

Chạy lại app

Console sẽ là:

```
🟡 1. BẮT ĐẦU gọi fetchItems
🟢 2. SAU khi Future.delayed xong
```

👉 **CHẠY LIỀN – KHÔNG ĐỢI**

GIẢI THÍCH LỖI NÀY

- Không `await` → Dart **không chờ**
- Future vẫn chạy, nhưng code **chạy tiếp**
- 👉 Dẫn tới:
  - dữ liệu sai
  - UI hiển thị trước khi có data

💋 **Đây là lỗi sinh viên gặp NHIỀU NHẤT**

⚠ LỖI THƯỜNG GẶP Ở BƯỚC 4

❌ Lỗi 1: Không thấy print

🔍 Nguyên nhân:

- Chưa chạy app
- Xem sai console

❌ Lỗi 2: Không thấy delay

🔍 Nguyên nhân:

- Quên `await`

PhucPT10@fpt.edu.vn

❌ Lỗi 3: Nghĩ rằng async là chạy song song

👄 SAI:

async ≠ đa luồng

💡 TIPS CHO SINH VIÊN

✅ Luôn đánh số print:

```
print('1...');
print('2...');
```

✅ Mỗi `await` = **1 điểm dừng logic**

✅ Không hiểu async → **in print ra xem thứ tự**

🎓 KẾT LUẬN BƯỚC 4

👉 Nhớ **DUY NHẤT 2 Ý**:

1️⃣ `await` **chỉ dừng hàm**, không dừng app

2️⃣ Quên `await` → code chạy **SAI THỨ TỰ**

◈ BƯỚC 5: TẠO HomeScreen (CHƯA ASYNC)

🎯 Mục tiêu

- Tách UI thành Screen
- CHƯA dùng FutureBuilder
- Tránh quá tải kiến thức

🛠 THAO TÁC

1️⃣ Chuột phải `lib`

```
New Folder → screens
```

2️⃣ Chuột phải `screens`

```
New File → home_screen.dart
```

📋 CODE: `lib/screens/home_screen.dart`

```
import 'package:flutter/material.dart';

class HomeScreen extends StatelessWidget {
  const HomeScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Async Product Loader'),
      ),
      body: const Center(
        child: Text('Chưa tải dữ liệu'),
      ),
    );
  }
}
```

◈ BƯỚC 6: GẮN FUTURE VÀO UI BẰNG FutureBuilder

🎯 Mục tiêu (QUAN TRỌNG NHẤT)

- UI **KHÔNG tạo Future**

PhucPT10@fpt.edu.vn

- UI **CHỈ LẮNG NGHE Future**
- Hiển thị đủ:
  - loading
  - data
  - error

🛠 THAO TÁC

1 MỞ FILE

```
lib/screens/home_screen.dart
```

2 IMPORT SERVICE & MODEL

```
import '../services/fake_api_service.dart';
import '../models/item.dart';
```

3 THAY body BẰNG FutureBuilder

```
body: FutureBuilder<List<Item>>(
  future: FakeApiService.fetchItems(),
  builder: (context, snapshot) {
```

4 XỬ LÝ LOADING (BẮT BUỘC)

```
if (snapshot.connectionState == ConnectionState.waiting) {
  return const Center(
    child: CircularProgressIndicator(),
  );
}
```

5 XỬ LÝ LỖI

```
if (snapshot.hasError) {
  return const Center(
    child: Text('Có lỗi xảy ra'),
  );
}
```

6 LẤY DATA & HIỂN THỊ

```
final items = snapshot.data!;

return ListView.builder(
  itemCount: items.length,
  itemBuilder: (context, index) {
    final item = items[index];
    return ListTile(
      title: Text(item.name),
      subtitle: Text('${item.price} VND'),
    );
  },
);
```

✅ FULL CODE BƯỚC 6 (ĐỂ SO SÁNH)

```
class HomeScreen extends StatelessWidget {
  const HomeScreen({super.key});

  @override
```


PhucPT10@fpt.edu.vn

```
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Async Product Loader')),
      body: FutureBuilder<List<Item>>(
        future: FakeApiService.fetchItems(),
        builder: (context, snapshot) {
          if (snapshot.connectionState == ConnectionState.waiting) {
            return const Center(
              child: CircularProgressIndicator(),
            );
          }

          if (snapshot.hasError) {
            return const Center(
              child: Text('Có lỗi xảy ra'),
            );
          }

          final items = snapshot.data!;

          return ListView.builder(
            itemCount: items.length,
            itemBuilder: (context, index) {
              final item = items[index];
              return ListTile(
                title: Text(item.name),
                subtitle: Text('${item.price} VND'),
              );
            },
          );
        },
      ),
    );
  }
}
```

🎓 TỔNG KẾT SAU BƯỚC 6 (CHỐT CHO SV)

👉 **Nếu chỉ nhớ 3 câu:**

1️⃣ Future **không trả dữ liệu ngay**

2️⃣ `await` **chỉ dừng hàm – không dừng app**

3️⃣ `FutureBuilder` = cầu nối **Future ↔ UI**

NHẬN DIỆN VẤN ĐỀ

◈ BƯỚC 6.1: MỞ ĐÚNG FILE UI

🗁 Thao tác

1. Mở file:

`lib/screens/home_screen.dart`

2. Đảm bảo đang có cấu trúc cơ bản:

```
class HomeScreen extends StatelessWidget {
  const HomeScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
```


PhucPT10@fpt.edu.vn

```
      appBar: AppBar(
        title: const Text('Async Product Loader'),
      ),
      body: Center(
        child: Text('Hello'),
      ),
    );
  }
}
```

👉 **Chưa dùng async ở đây**

◈ BƯỚC 6.2: XÓA UI CŨ – CHUẨN BỊ GẮN FUTUREBUILDER

❌ XÓA đoạn này:

```
Center(
  child: Text('Hello'),
),
```

✅ Thay bằng khung rỗng để dễ nhìn:

```
body: Container(),
```

👉 Việc này giúp sinh viên **biết rõ mình đang thay UI**

◈ BƯỚC 6.3: NHẬP FUTUREBUILDER (KHÔNG VIẾT VỘI)

✍ Gõ từ từ:

```
body: FutureBuilder(
```

👉 Dừng lại – GIẢI THÍCH:

- `FutureBuilder` là **Widget**
- Nó **KHÔNG chạy async**
- Nó **CHỈ LẮNG NGHE kết quả Future**

◈ BƯỚC 6.4: TRUYỀN FUTURE VÀO FUTUREBUILDER

✍ Viết tiếp:

```
body: FutureBuilder(
  future: FakeApiService.fetchItems(),
```

🧠 GIẢI THÍCH RÕ:

- `fetchItems()` trả về `Future<List<Item>>`
- FutureBuilder **giữ Future này**
- Flutter **tự động rebuild UI** khi Future xong

⚠ NHẤN MẠNH:
"Future chạy ở SERVICE
UI chỉ nhận kết quả"

◈ BƯỚC 6.5: VIẾT builder – PHẦN SINH VIÊN DỄ SAI NHẤT

✍ Viết tiếp:

```
builder: (context, snapshot) {
```

👉 DỪNG LẠI – GIẢI THÍCH snapshot:

- `snapshot` = **ảnh chụp trạng thái Future**
- snapshot có thể:
  - đang chờ
  - có dữ liệu
  - có lỗi

PhucPT10@fpt.edu.vn

◈ BƯỚC 6.6: XỬ LÝ TRẠNG THÁI LOADING (BẮT BUỘC)

✍ Viết:

```
if (snapshot.connectionState == ConnectionState.waiting) {
  return const Center(
    child: CircularProgressIndicator(),
  );
}
```

🧠 GIẢI THÍCH CHẬM:

- `waiting` = Future **chưa xong**
- UI **PHẢI HIỂN THỊ LOADING**
- Nếu không → UX rất kém

👄 LỖI RẤT HAY GẶP:

"App không hiện gì → sinh viên tưởng app lỗi"

◈ BƯỚC 6.7: XỬ LÝ LỖI FUTURE

✍ Viết tiếp:

```
if (snapshot.hasError) {
  return Center(
    child: Text('Có lỗi xảy ra: ${snapshot.error}'),
  );
}
```

🧠 GIẢI THÍCH:

- Future **có thể fail**
- Nếu không xử lý → crash hoặc trắng màn hình

◈ BƯỚC 6.8: LẤY DỮ LIỆU KHI FUTURE THÀNH CÔNG

✍ Viết:

```
final items = snapshot.data!;
```

👉 GIẢI THÍCH:

- `snapshot.data` chỉ có **SAU khi Future xong**
- Dùng `!` vì:
  - đã qua waiting
  - đã qua error

◈ BƯỚC 6.9: HIỂN THỊ DỮ LIỆU RA UI

✍ UI đơn giản nhất:

```
return ListView.builder(
  itemCount: items.length,
  itemBuilder: (context, index) {
    final item = items[index];
    return ListTile(
      title: Text(item.name),
      subtitle: Text('${item.price} VND'),
    );
  },
);
```

👉 KHÔNG style – KHÔNG tối ưu – CHỈ TẬP TRUNG ASYNC

◈ BƯỚC 6.10: TỔNG HỢP TOÀN BỘ FutureBuilder

👉 **Cho sinh viên SO SÁNH với code của mình**

```
body: FutureBuilder<List<Item>>(
```


PhucPT10@fpt.edu.vn

```
  future: FakeApiService.fetchItems(),
  builder: (context, snapshot) {
    if (snapshot.connectionState == ConnectionState.waiting) {
      return const Center(
        child: CircularProgressIndicator(),
      );
    }

    if (snapshot.hasError) {
      return Center(
        child: Text('Có lỗi xảy ra'),
      );
    }

    final items = snapshot.data!;

    return ListView.builder(
      itemCount: items.length,
      itemBuilder: (context, index) {
        final item = items[index];
        return ListTile(
          title: Text(item.name),
          subtitle: Text('${item.price} VND'),
        );
      },
    );
  },
),
```

🔍 KIỂM TRA KẾT QUẢ (CHECKPOINT BẮT BUỘC)

Khi chạy app:

✅ Thấy vòng tròn loading ~2 giây
✅ Sau đó hiện danh sách
✅ UI **KHÔNG bị đơ**

⚠️ LỖI PHỔ BIẾN Ở BƯỚC 6

❌ Lỗi 1: Gọi fetchItems trong build nhiều lần

🔍 Nguyên nhân:

- Không hiểu build chạy lại

🛠 Cách sửa (nói trước):

- Chuyển Future vào biến (sẽ học ở bước sau)

❌ Lỗi 2: snapshot.data null

🔍 Nguyên nhân:

- Không kiểm tra waiting
- Không kiểm tra error

❌ Lỗi 3: Không thấy loading

🔍 Nguyên nhân:

- Future chạy quá nhanh
- Hoặc quên CircularProgressIndicator

PhucPT10@fpt.edu.vn

🎓 KẾT LUẬN BƯỚC 6 (CHỐT TƯ DUY)

👉 Nếu sinh viên chỉ nhớ **3 Ý**:

1️⃣ UI Flutter **không chờ dữ liệu**

2️⃣ FutureBuilder = **cầu nối Future ↔ UI**

3️⃣ Phải xử lý đủ: **loading – data – error**

🔷 BƯỚC 7: Chạy ứng dụng & Quan sát

🧪 Quan sát UI

1. Mở app → **Loading xuất hiện**
2. Chờ 2 giây
3. Danh sách sản phẩm hiển thị

🧪 Quan sát Debug Console

```
⏳ Bắt đầu gọi fake API
(sau 2s)
✅ Fake API trả dữ liệu
```

👉 **Checkpoint quan trọng**

Nếu KHÔNG thấy loading → bạn đang sai tư duy async

7️⃣ KIỂM TRA & CHẠY ỨNG DỤNG

```
flutter run
```

Test:

- hot reload
- restart app
- xem Future có chạy lại không

8️⃣ CÁC LỖI PHỔ BIẾN VỚI FUTURE

❌ LỖI 1: Gọi Future trong build nhiều lần

Nguyên nhân

- build() chạy lại
- Future bị gọi lại

Cách sửa

- Gán Future vào biến
- Hoặc StatefulWidget (ở bài sau)

❌ LỖI 2: Không có loading

Nguyên nhân

- Không kiểm tra `ConnectionState.waiting`

Hậu quả

- UX rất tệ

❌ LỖI 3: snapshot.data null

Nguyên nhân

- Không kiểm tra `hasData`

Cách phòng tránh

- Luôn kiểm tra thứ tự:
    1. waiting
    2. error
    3. data

PhucPT10@fpt.edu.vn

9 KẾT QUẢ ĐẠT ĐƯỢC SAU DỰ ÁN

Sinh viên:

✅ Hiểu rõ:

- Future chạy **1 lần**
- async **không block UI**

✅ Biết dùng:

- `FutureBuilder`
- `async / await` đúng chỗ

✅ Sẵn sàng học:

- Stream
- API thật
- State Management

10 BÀI TẬP MỞ RỘNG (KHÔNG STREAM)

1 Thêm nút **Reload Future**

2 Thay `Future.delayed` bằng API thật (gợi ý)

3 Thay đổi delay 1s – 5s để quan sát UX

PhucPT10@fpt.edu.vn

BÀI TẬP TỔNG HỢP 2

1 TỔNG QUAN DỰ ÁN

Ứng dụng Flutter mô phỏng **dữ liệu thay đổi liên tục theo thời gian** (giống realtime):

- Mỗi 1 giây → số lượng sản phẩm tăng lên
- UI tự động cập nhật **KHÔNG reload**
- Sinh viên **NHÌN THẤY Stream đang chạy**

KHÔNG API – KHÔNG Firebase – KHÔNG Socket

Chỉ tập trung **bản chất Stream**

2 SO SÁNH NHANH (NHẮC LẠI KIẾN THỨC CŨ)

Future	Stream
Trả 1 lần	Trả nhiều lần
Dùng cho API	Dùng cho realtime
`FutureBuilder`	`StreamBuilder`
Hoàn thành xong là hết	Chạy liên tục

CÂU CHỐT CHO SV

Future = "đợi xong rồi lấy"
Stream = "có gì mới là báo liền"

3 CẤU TRÚC THƯ MỤC DỰ ÁN

```
lib/
├── main.dart
├── screens/
│   └── counter_stream_screen.dart
├── services/
│   └── counter_stream_service.dart
```

KHÔNG model – KHÔNG phức tạp

4 BƯỚC 1: TẠO PROJECT STREAM (VS CODE)

Mục tiêu

- Tạo app Flutter mới
- Tránh lẫn với Future

THAO TÁC

1 Mở VS Code

2 Mở Terminal

```
Ctrl + `
```

3 Tạo project

```
flutter create stream_counter_app
```

4 Open Folder

```
File → Open Folder → stream_counter_app
```

5 BƯỚC 2: DỌN main.dart (TỐI GIẢN)

MỞ FILE

```
lib/main.dart
```

XÓA TOÀN BỘ → DÁN CODE

```
import 'package:flutter/material.dart';
import 'screens/counter_stream_screen.dart';

void main() {
```


PhucPT10@fpt.edu.vn

```
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      home: CounterStreamScreen(),
    );
  }
}
```

▶ CHẠY THỬ

```
flutter run
```

⚠️ **LỖI DỰ KIẾN**

Chưa có `CounterStreamScreen`

👉 **ĐÚNG – chuyển bước tiếp**

6 BƯỚC 3: TẠO STREAM SERVICE (QUAN TRỌNG NHẤT)

🎯 Mục tiêu

- Tạo `StreamController`
- Phát dữ liệu liên tục
- TÁCH LOGIC KHỎI UI

🛠 THAO TÁC

1 Tạo thư mục

```
lib/services
```

2 Tạo file

```
counter_stream_service.dart
```

📋 CODE: `lib/services/counter_stream_service.dart`

```
import 'dart:async';

class CounterStreamService {
  final StreamController<int> _controller =
      StreamController<int>();

  int _count = 0;

  CounterStreamService() {
    _startCounting();
  }

  Stream<int> get stream => _controller.stream;

  void _startCounting() {
    Timer.periodic(const Duration(seconds: 1), (timer) {
      _count++;
      print('🔵 Stream phát giá trị: $_count');
      _controller.sink.add(_count);
```


PhucPT10@fpt.edu.vn

```
    });
  }

  void dispose() {
    _controller.close();
  }
}
```

🧠 GIẢI THÍCH CỰC CHẬM (BẮT BUỘC)

◈ StreamController

- Là **nguồn phát dữ liệu**
- Chủ động `add()` giá trị

◈ sink.add()

- Đẩy dữ liệu mới vào Stream
- Mỗi lần add → UI được báo

◈ stream

- UI **chỉ được nghe**
- UI **KHÔNG được add**

👄 **CÂU CHỐT**

Service phát – UI nghe

7️⃣ BƯỚC 4: TẠO SCREEN HIỂN THỊ STREAM

🎯 Mục tiêu

- Dùng `StreamBuilder`
- Quan sát dữ liệu thay đổi liên tục

🛠️ THAO TÁC

1️⃣ Tạo file

`lib/screens/counter_stream_screen.dart`

📋 CODE: `counter_stream_screen.dart`

```
import 'package:flutter/material.dart';
import '../services/counter_stream_service.dart';

class CounterStreamScreen extends StatefulWidget {
  const CounterStreamScreen({super.key});

  @override
  State<CounterStreamScreen> createState() =>
      _CounterStreamScreenState();
}
```

8️⃣ BƯỚC 5: KẾT NỐI STREAM VỚI UI

🎯 Mục tiêu

- Tạo service
- Gắn StreamBuilder
- Xử lý trạng thái

📋 CODE TIẾP

```
class _CounterStreamScreenState
    extends State<CounterStreamScreen> {
```

```
  late CounterStreamService _service;

  @override
  void initState() {
    super.initState();
    _service = CounterStreamService();
  }

  @override
  void dispose() {
    _service.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Stream Counter'),
      ),
      body: Center(
        child: StreamBuilder<int>(
          stream: _service.stream,
          builder: (context, snapshot) {
```

9 BƯỚC 6: XỬ LÝ TRẠNG THÁI STREAM (RẤT QUAN TRỌNG)

✍ VIẾT TIẾP

```
if (!snapshot.hasData) {
  return const CircularProgressIndicator();
}

return Text(
  'Số lượng: ${snapshot.data}',
  style: const TextStyle(fontSize: 32),
);
```

✅ FULL builder

```
builder: (context, snapshot) {
  if (!snapshot.hasData) {
    return const CircularProgressIndicator();
  }

  return Text(
    'Số lượng: ${snapshot.data}',
    style: const TextStyle(fontSize: 32),
  );
},
```

▶ CHẠY ỨNG DỤNG & QUAN SÁT

```
flutter run
```

👀 KẾT QUẢ

- Mỗi 1 giây → số tăng
- UI cập nhật liên tục

PhucPT10@fpt.edu.vn

- KHÔNG reload
- Console in log Stream

⚠ NHẬN DIỆN LỖI PHỔ BIẾN (BẮT BUỘC DẠY)

❌ Lỗi 1: Không dùng StatefulWidget

👄 Không dispose → rò rỉ bộ nhớ

❌ Lỗi 2: Không đóng StreamController

👄 App chạy lâu → crash

❌ Lỗi 3: Nghĩ Stream = đa luồng

👄 SAI

Stream = dữ liệu theo thời gian

🎓 TỔNG KẾT TƯ DUY (CHỐT CHO SV MỚI)

👉 **Nếu chỉ nhớ 4 câu:**

1️⃣ Stream phát nhiều lần

2️⃣ StreamController là nguồn

3️⃣ StreamBuilder là UI listener

4️⃣ Dispose là BẮT BUỘC

🧪 BÀI TẬP CỦNG CỐ (KHÔNG STREAM NÂNG CAO)

1️⃣ Thêm nút **Pause / Resume Stream**

2️⃣ Thay số đếm bằng **giá sản phẩm realtime**

3️⃣ Đổi thời gian 1s → 500ms → quan sát

PhucPT10@fpt.edu.vn

10 PROMPT TỰ NGHIÊN CỨU CHUYÊN SÂU

10 PROMPT PHÂN TÍCH – ĐÁNH GIÁ (LEVEL KHÁ – GIỎI)

Sinh viên đã:

- Học xong **kiến thức đầy đủ Slot 4** (Prompt 4)
- Được cảnh báo lỗi & best practices (Prompt 5)

Giờ đây, sinh viên cần:

- Tự nghiên cứu **sâu hơn mức bắt buộc**
- Phát triển tư duy:
  - Phân tích
  - So sánh
  - Đánh giá
  - Lựa chọn giải pháp

👉 Các prompt dưới đây được thiết kế để sinh viên:

- **Tự đặt câu hỏi đúng**
- **Tự tranh luận với AI**
- **Chuẩn bị nền tảng cho project Flutter lớn hơn**

YÊU CẦU CHUNG CHO TẤT CẢ PROMPT

Khi trả lời mỗi prompt, sinh viên **bắt buộc** phải:

- Giải thích **vì sao chọn giải pháp**
- Chỉ ra **ưu / nhược điểm**
- Nêu **khi nào nên / không nên áp dụng**
- Liên hệ đến **ứng dụng Flutter thực tế**

Prompt 1 – Tư duy bất đồng bộ

Bối cảnh:
Một sinh viên nghĩ rằng `async/await` giúp code "chạy nhanh hơn".
Câu hỏi:
Phân tích nhận định trên. Trong trường hợp nào async giúp app mượt hơn nhưng không làm code chạy nhanh hơn?
Mục tiêu:
Hiểu đúng bản chất async ≠ song song.

Prompt 2 – Future & Thứ tự thực thi

Bối cảnh:
Một app gọi API, nhưng dữ liệu hiển thị sai thứ tự.
Câu hỏi:
Phân tích nguyên nhân liên quan đến `Future` và `await`. Nếu bỏ `await`, chuyện gì xảy ra?
Mục tiêu:
Hiểu luồng thực thi async.

Prompt 3 – Error Handling trong async

Bối cảnh:
App không crash nhưng UI không cập nhật khi API lỗi.
Câu hỏi:
Phân tích cách lỗi async bị "nuốt mất". So sánh `try-catch` với `.catchError()`.
Mục tiêu:
Nâng cao tư duy xử lý lỗi async.

PhucPT10@fpt.edu.vn

◈ Prompt 4 – Future vs Stream

Bối cảnh:
Một nhóm dùng `Future` cho dữ liệu realtime.
Câu hỏi:
Đánh giá quyết định này. Khi nào `Stream` là lựa chọn bắt buộc?
Mục tiêu:
Biết chọn đúng async pattern.

◈ Prompt 5 – Stream & Memory Leak

Bối cảnh:
App chạy lâu bị chậm dần.
Câu hỏi:
Phân tích vai trò của `StreamController` và việc không đóng stream.
Mục tiêu:
Hiểu rủi ro memory leak.

◈ Prompt 6 – Single vs Broadcast Stream

Bối cảnh:
Một stream được nhiều widget lắng nghe.
Câu hỏi:
So sánh single-subscription và broadcast stream. Sai lầm phổ biến khi dùng broadcast?
Mục tiêu:
Hiểu sâu stream behavior.

◈ Prompt 7 – Async & UI Flutter

Bối cảnh:
Dữ liệu đã load xong nhưng UI không cập nhật.
Câu hỏi:
Phân tích mối liên hệ giữa async code và vòng đời widget.
Mục tiêu:
Kết nối async với UI & state.

◈ Prompt 8 – Hiệu năng async

Bối cảnh:
Một màn hình có quá nhiều Future được gọi cùng lúc.
Câu hỏi:
Phân tích tác động đến hiệu năng và trải nghiệm người dùng.
Mục tiêu:
Biết giới hạn async.

◈ Prompt 9 – Tổ chức async code

Bối cảnh:
Async code nằm rải rác trong widget.
Câu hỏi:
Đánh giá rủi ro maintain & test. Đề xuất cách tổ chức tốt hơn.
Mục tiêu:
Tư duy clean code & kiến trúc.

◈ Prompt 10 – Tư duy chuyên nghiệp

Bối cảnh:
Hai cách xử lý async đều chạy đúng.

PhucPT10@fpt.edu.vn

Câu hỏi:
Dựa vào tiêu chí nào để chọn giải pháp tốt hơn trong dự án thật?
Mục tiêu:
Hình thành tư duy kỹ sư phần mềm.

10 PROMPT TỰ NGHIÊN CỨU THỰC TIỄN CHUYÊN NGHIỆP

1 10 PROMPT THỰC TIỄN CHUYÊN NGHIỆP (PROJECT-SCALE THINKING)

Mỗi prompt buộc sinh viên phải:

- Phân tích
- Đánh giá rủi ro
- Đề xuất hướng giải quyết
- Liên hệ môi trường làm việc thật

◈ Prompt 1 – Async trong app lớn

Bối cảnh: App Flutter có hàng chục API call song song.
Vấn đề: UI giật lag khi load dữ liệu.
Câu hỏi: Phân tích vai trò async trong vấn đề này và đề xuất chiến lược xử lý.
Năng lực: Tư duy hiệu năng.

◈ Prompt 2 – Future chaining phức tạp

Bối cảnh: Một luồng xử lý gồm nhiều Future phụ thuộc nhau.
Vấn đề: Code khó đọc, khó debug.
Câu hỏi: Đánh giá các cách tổ chức async flow và rủi ro maintain.
Năng lực: Clean async code.

◈ Prompt 3 – Async & trải nghiệm người dùng

Bối cảnh: Màn hình load chậm nhưng không có loading UI.
Vấn đề: Người dùng tưởng app bị treo.
Câu hỏi: Async ảnh hưởng UX thế nào? Tech Lead cần quyết định gì?
Năng lực: UX + async.

◈ Prompt 4 – Stream trong realtime feature

Bối cảnh: App chat dùng Stream để nhận tin nhắn.
Vấn đề: App tốn pin và RAM.
Câu hỏi: Đánh giá cách dùng Stream & StreamController.
Năng lực: Realtime optimization.

◈ Prompt 5 – Stream lifecycle

Bối cảnh: App crash sau khi navigate nhiều màn hình.
Vấn đề: Stream không được dispose.
Câu hỏi: Phân tích hậu quả trong dự án lớn.
Năng lực: Memory management.

◈ Prompt 6 – Async error trong production

Bối cảnh: App lỗi ngẫu nhiên, không tái hiện được.
Vấn đề: Lỗi async không được log đầy đủ.
Câu hỏi: Tech Lead cần thiết kế error handling async thế nào?
Năng lực: Stability & logging.

◈ Prompt 7 – Retry strategy

Bối cảnh: API không ổn định.
Vấn đề: Gọi lại quá nhiều lần gây quá tải.

PhucPT10@fpt.edu.vn

Câu hỏi: Phân tích chiến lược retry hợp lý.
Năng lực: Network resilience.

◈ Prompt 8 – Async & testing
Bối cảnh: Unit test async chạy không ổn định.
Vấn đề: Test flaky.
Câu hỏi: Async ảnh hưởng test thế nào và cách thiết kế tốt hơn?
Năng lực: Testability.

◈ Prompt 9 – Async trong Clean Architecture
Bối cảnh: Async logic nằm trực tiếp trong UI layer.
Vấn đề: Code khó mở rộng.
Câu hỏi: Đánh giá và đề xuất tổ chức async theo layer.
Năng lực: Kiến trúc phần mềm.

◈ Prompt 10 – Stream vs polling
Bối cảnh: App cập nhật dữ liệu liên tục.
Vấn đề: Không biết dùng Stream hay gọi API định kỳ.
Câu hỏi: Phân tích trade-off hai cách tiếp cận.
Năng lực: Quyết định kiến trúc.

◈ Prompt 11 – Async & state consistency
Bối cảnh: Dữ liệu hiển thị không đồng bộ giữa các màn hình.
Vấn đề: Async race condition.
Câu hỏi: Phân tích nguyên nhân và cách phòng tránh.
Năng lực: Data consistency.

◈ Prompt 12 – Performance audit
Bối cảnh: App chậm trên máy cấu hình yếu.
Vấn đề: Async tasks chạy không kiểm soát.
Câu hỏi: Làm sao audit và tối ưu async performance?
Năng lực: Performance tuning.

◈ Prompt 13 – Async & team collaboration
Bối cảnh: Nhiều dev viết async theo cách khác nhau.
Vấn đề: Codebase thiếu nhất quán.
Câu hỏi: Tech Lead cần đặt rule async gì cho team?
Năng lực: Team-level engineering.

◈ Prompt 14 – Migration async code
Bối cảnh: Dự án cũ dùng callback-style.
Vấn đề: Khó nâng cấp.
Câu hỏi: Phân tích chiến lược migrate sang async/await.
Năng lực: Refactoring.

◈ Prompt 15 – Tư duy senior
Bối cảnh: Hai giải pháp async đều chạy đúng.
Vấn đề: Phải chọn một cho dự án 3 năm tới.
Câu hỏi: Senior Flutter Developer sẽ dựa vào tiêu chí nào?
Năng lực: Tư duy dài hạn.

PhucPT10@fpt.edu.vn